

AI Debugger Pro

Paper 0 - From Creative-System Failure to Evidence-Governed AI-Assisted Repair

T. R. Bentley
Foundation and Series Overview
September 2026

Abstract

AI Debugger Pro began as a practical response to a failure encountered while attempting to build a broader AI-driven creative production system. Fragmented tools, external dependencies, cost, and orchestration problems made the debugging layer itself the bottleneck. The project therefore inverted its original priority: before a creative system could become increasingly autonomous, its software substrate needed a disciplined way to execute, observe, diagnose, propose repairs, preserve history, and verify change. This paper reconstructs that origin and places the resulting debugger in its current research context. It preserves the early V0-V4 development history while separating historical behavior from the current architecture and from later design proposals. The current research series spans system evolution, human-governed repair, security, verification, multi-language execution, human factors, empirical benchmarking, failure taxonomy, context engineering, independent test oracles, and reproducible release evidence. Measured claims are deliberately bounded: the frozen execution benchmark contains 24 cases with 24 expected classifications matched; 14 of 16 observed failures returned non-empty diagnostic evidence; and the recorded CI baseline passed 27 tests on each of Python 3.10 and 3.12. These measurements establish an execution and evidence baseline, not proof of autonomous repair effectiveness.

Paper role. Paper 0 is the entry point to the twelve-paper technical series. It is a retrospective foundation paper, not a replacement for the detailed evidence and protocols in Papers 1-12.

1. Origin: the debugger was not the original project

The project began somewhere else. The original objective was an end-to-end AI-driven creative production studio capable of coordinating narrative media workflows such as scripts, music, and visual content. Early designs relied on assembling existing AI services into a larger pipeline. That approach exposed a systems problem: the creative goal was increasingly constrained by fragmented tools, external dependencies, cost, and the absence of a unified orchestration layer that could diagnose failures across the system.

That failure changed the research question. Instead of asking only how to generate creative output, the project began asking what infrastructure would be required for a larger AI system to inspect and repair the software on which it depended. AI Debugger Pro emerged from that inversion. Debugging stopped being a supporting utility and became foundational infrastructure for the broader autonomy problem.

This origin matters because it explains several later design choices. Multi-language execution follows from the need to operate across heterogeneous components. History and diffs follow from the need to understand change over time. Human approval follows from the danger of allowing probabilistic suggestions to mutate source automatically. Context selection, independent oracles, and reproducibility follow from a later realization: a repair system is not credible merely because it produces plausible code. It needs evidence.

Connection to the creative-system project. The debugger is therefore both an independent software-engineering project and infrastructure born from a separate creative-system ambition. Paper 0 intentionally preserves that cross-project connection rather than sanitizing the origin into a generic debugging narrative.

1.1 Research reframing

The project can be summarized as a sequence of reframings: creative automation exposed orchestration failure; orchestration failure exposed debugging as a bottleneck; AI-assisted debugging exposed trust and verification problems; and those trust problems produced the present evidence-governed research program.

2. Historical evolution: V0-V6

The early paper documented V0-V4 as the system changed from a stateless Python runner into an interactive, multi-language, AI-assisted debugger with temporal state. The later research series extends that history through the human-governed repair architecture. The table below keeps historical capabilities distinct from current claims.

Version	Primary shift	Research significance
V0	Python syntax validation + temporary-file execution	Deterministic, stateless baseline
V1	LLM-assisted error interpretation and correction suggestions	Hybrid deterministic/probabilistic debugging
V2	Python, C, and Java execution routing	Language-specific adapters and orchestration
V3	Desktop GUI; C++ added to active execution platform	Interactive debugging loop
V4	History, diffs, real-time syntax feedback, layered temporal state	Temporal state and event-driven feedback
V5	Structured diagnosis and stronger repair workflow	Suggestions become inspectable repair proposals
V6	Approval-gated mutation, rerun, evidence-oriented repair	Iterative, verified repair protocol

Historical precision

V0-V4 should be read as development states, not retroactively upgraded descriptions. For example, the original V0 execution model relied on subprocess execution and temporary files; later container isolation and governance controls should not be projected backward onto that prototype. Likewise, early qualitative observations are preserved as historical development notes rather than reused as modern empirical claims.

3. Current system model

The current project is best understood as a human-governed AI-assisted repair system built around deterministic execution evidence. Its architecture separates what the machine observed from what the language model inferred, and separates what the model proposed from what the user authorized.

Layer	Current role	Trust boundary
Execution	Run supported code; capture success/failure, output, return state	Observed execution evidence is not an AI judgment
Analysis / diagnostics	Normalize syntax, compile and runtime evidence	Diagnostic text can be incomplete or ambiguous
AI diagnosis	Produce structured root-cause interpretation and candidate corrections	Probabilistic; must not claim verification
Repair proposal	Present corrected code/diff before mutation	Proposal is not source state
Human approval	Explicit approve/reject decision before source mutation	Governance boundary
Verification	Rerun after approved change; preserve result and evidence	Passing scoped checks does not prove global correctness
History / provenance	Record executions, changes, diffs and relationships	Supports auditability and reconstruction
Project context	Bound multi-file source context under declared limits	Selection can omit relevant evidence or expose risky context

Supported execution baseline. The repository research series treats Python, C, C++, and Java as the supported multi-language execution set. Docker is the default execution backend in the current architecture, while local execution is an explicit opt-in path. No sandbox is absolute, so isolation is treated as risk reduction rather than a proof of safety.

4. Human-governed repair protocol

A central architectural change from the early paper is the treatment of repair as a governed state transition rather than a model-generated answer. The intended lifecycle is:

1	Observe	Execute or validate code and capture structured failure evidence.
2	Diagnose	Use deterministic evidence plus bounded context to produce a structured AI interpretation.
3	Propose	Generate candidate corrected code and explanation without mutating source.
4	Review	Show the proposal/diff and supporting evidence to the user.
5	Authorize	Require explicit approval before source mutation.
6	Apply	Write the approved change while retaining provenance.
7	Verify	Rerun relevant checks and record scoped outcomes.
8	Preserve	Store history, diffs, evidence, and relationships for later audit.

Why the approval gate matters

The LLM is allowed to interpret and propose, but the user remains the authority over source mutation. This boundary reduces the risk that fluent but incorrect reasoning silently becomes repository state. The rerun after approval then produces new evidence; it does not retroactively make the model's explanation true.

5. Measured baseline: what has actually been demonstrated

The original paper used qualitative functional observations. The modern series replaces those broad statements with a frozen, explicitly scoped execution benchmark and recorded CI evidence.

Measured item	Result	Interpretation
Frozen execution cases	24 attempted; 24 expected classification	100% agreed on this controlled corpus
Observed failure cases	16	Subset used for diagnostic-evidence coverage
Non-empty failure evidence	14 of 16	87.5% textual evidence coverage; non-empty does not mean actionable
Overall median execution latency	282.9685 ms	Single recorded environment; not cross-machine performance
Overall mean execution latency	351.7978 ms	Same scope limitation
CI - Python 3.10	27 tests passed; 1.12 s	Recorded repository baseline
CI - Python 3.12	27 tests passed; 1.17 s	Recorded repository baseline

Claim boundary. These results measure execution classification, latency, diagnostic-evidence presence, and CI status. They do not establish LLM repair accuracy, autonomous debugging effectiveness, security completeness, or generalization to arbitrary repositories.

5.1 The silent native-exit gap

Two native runtime failures were correctly classified as failures but returned empty textual diagnostics. This finding motivated a broader evidence model in which return code, signal, phase, timeout state, stdout/stderr presence, backend, language/toolchain, and bounded trace information are preserved independently rather than collapsing failure evidence into one string.

6. Evidence maturity: measured, implemented, specified, planned

A recurring lesson of the series is that software papers become misleading when architecture, measurements, and aspirations are written in the same grammatical tense. Paper 0 therefore adopts four evidence labels.

Label	Meaning	Examples in this project
MEASURED	Observed under a recorded procedure	24-case execution benchmark; diagnostic-evidence coverage; CI baseline
IMPLEMENTED	Present in repository behavior or workflow	Multi-language runners; Docker-default path; approval-before-mutation work
DESIGN SPECIFICATION	Defined architecture or frozen validation scenario	Control-systems model; error-detection; admission/oracle framework
PLANNED / HYPOTHESIS	Future outcome requiring new evidence	Diagnosis lift from improved context; safety gains from generated-test oracles

This distinction also resolves a weakness in the historical document. Early statements such as AI suggestions being accurate for common errors are useful development observations, but they were not accompanied by a frozen LLM evaluation protocol, blinded labels, or repair-accuracy dataset. They should not be promoted into modern measured claims.

7. Research series map: Papers 1-6

Paper 0 provides the narrative spine. Papers 1-12 decompose the system into focused research questions.

Paper	Focus	Role in the evidence program
1 - System Evolution	V0-V6 architectural progression	Detailed historical case study
2 - Current Architecture	Human-governed repair protocol	Authoritative current system model
3 - Evaluation Protocol	Empirical repair-study design	Defines how stronger effectiveness claims should be tested
4 - Security Threat Model	Execution, context, prompt and repair risk	Defines adversaries, trust boundaries and mitigations
5 - Repair Verification and Evidence	Evidence labels, provenance and verification	Prevents 'it ran' from becoming 'it is correct'
6 - Multi-Language Execution	Shared lifecycle vs language-specific semantics	Separates common orchestration from Python/C/C++/Java behavior

8. Research series map: Papers 7-12

Paper	Focus	Role in the evidence program
7 - Human-in-the-Loop UX	Decision support and automation-bias protection	Tests whether governance interfaces improve human decisions
8 - Benchmark Results	Frozen execution benchmark	First measured multi-language baseline
9 - Failure Taxonomy	Execution-to-governance failure layers	Turns observed and prospective failures into a typed vocabulary
10 - Context Selection	Diagnostic relevance, risk and bounded context	Defines context as governed evidence rather than prompt bulk
11 - Regression-Test Oracle	Oracle independence and test admission	Separates generated challenges from authoritative verification
12 - Reproducible Engineering	Artifacts, claims, release gates and replication	Binds the series to durable, inspectable evidence

Together, the sequence moves from what the system became, through how it should be evaluated and governed, to what has been measured, and finally to how claims and artifacts should remain reproducible.

9. Context is evidence, not merely prompt material

The move from single-file debugging to project-aware diagnosis creates a new systems problem: not every available file should be sent to an LLM, and more context is not automatically better. Context can be irrelevant, stale, privacy-sensitive, secret-bearing, or adversarial.

Paper 10 therefore treats context selection as a governed pipeline: normalize direct failure evidence; generate candidate context; filter risk; rank relevance; allocate a deterministic budget; review where required; and bind the result to provenance. Its source matrix prioritizes direct failure evidence and focal source, treats bounded runtime locals as high-value but high-privacy evidence, and excludes credentials, full environment dumps, generated caches, and repository internals by default.

Important status. The current Paper 10 repository tables and figures are design specifications and frozen validation scenarios. They are not evidence that a proposed selector has already improved LLM diagnosis.

10. Verification requires independent authority

A generated regression test can be useful while still sharing the same misunderstanding as the repair that produced it. Paper 11 therefore ranks oracle sources by independence and defines an admission pipeline rather than treating any green generated test as proof.

The framework distinguishes candidate self-reruns, same-response generated tests, separately generated challenges, specification-derived tests, repository-owned preexisting tests, independently curated hidden tests, properties or metamorphic relations, differential references, formal contracts, security policies, human-adjudicated requirements, and production invariants. Stronger verification claims require stronger and more independent authority.

11. Security and governance

Executing arbitrary code and sending debugging context to an AI model create two different security surfaces. Execution isolation reduces the blast radius of untrusted code. Context governance reduces disclosure and prompt-injection risk. Repair governance reduces the chance that a plausible but unsafe patch becomes source state.

Risk surface	Representative failure	Control direction
Execution	Untrusted code accesses network, filesystem or excessive resources	Container isolation, bounded resources, explicit local opt-in
Diagnostic evidence	Sensitive locals or environment data enter model context	Redaction, bounded traces, source typing, preview
Prompt injection	Repository text attempts to act as model instruction	Treat project text as untrusted evidence, not instruction authority
Repair	Candidate patch changes unrelated or security-sensitive behavior	Diff review, explicit approval, risky-change warnings
Verification	Model-generated test ratifies its own misunderstanding	Independent oracles, preserved original tests, mutation challenges
Provenance	Evidence cannot be tied to source state	Commit/hash binding, immutable records, release manifests

Security is therefore not a single sandbox feature. It is a chain of controls spanning execution, evidence selection, model authority, human authorization, verification, and provenance.

12. Reproducible engineering as the endpoint

Paper 12 closes the series by treating publication itself as an engineering system. A credible claim should be traceable through an evidence graph: source state -> environment -> frozen inputs -> raw observations -> derivation procedure -> published claim -> release artifact -> replication record.

The repository's claim-evidence matrix already distinguishes measured execution results from unmeasured hypotheses. For example, the execution benchmark and diagnostic-evidence counts are measured; multi-language execution and the Docker-default backend are implemented; diagnosis improvement from a future context selector is unmeasured; generated-test safety improvement is a hypothesis; and independent replication remains planned.

This is the major methodological difference between the historical paper and Paper 0. The early document asked whether the system appeared to work across representative examples. The modern series asks what exact claim is being made, what artifact supports it, under what environment, with what limitations, and whether another evaluator can reconstruct the chain.

13. Limitations and open research questions

The sophistication of the architecture should not be confused with completeness of empirical validation. Several important questions remain open.

Area	Current limitation / open question
Repair effectiveness	The measured benchmark does not score LLM diagnosis correctness or repair success.
Corpus scale	The frozen execution benchmark is small and researcher-authored.
Generalization	Current measured results do not establish performance on arbitrary real-world repositories.
Context effectiveness	Paper 10 specifies selection and risk controls; diagnosis lift still requires a comparative experiment.
Oracle effectiveness	Paper 11 specifies admission and independence rules; unsafe-acceptance reduction remains to be measured.
Human factors	The UX study protocol exists, but user-study outcomes require actual participants and recorded analysis.
Security	Container controls reduce risk but cannot establish absolute sandbox safety.
Replication	Independent external replication has not yet been archived.

These are not defects in the paper's framing; they are the boundaries that keep architectural maturity from being mistaken for experimental proof.

14. Research trajectory beyond the twelve-paper foundation

The next stage should convert the strongest design hypotheses into comparative experiments. Three lines are especially important:

Context-selection effectiveness. Compare the current deterministic bounded project context against relevance-aware selectors using frozen multi-file defects. Score root-cause localization, diagnosis correctness, repair success, token/context cost, privacy exposure, and abstention quality.

Independent repair verification. Evaluate candidate repairs against preexisting tests, hidden tests, properties, mutation challenges, and security-policy checks. Measure how often weak or same-response generated tests would have accepted an unsafe or overfit repair.

Multi-agent debugging. Only after evidence contracts are strong enough should multiple specialized agents be allowed to coordinate diagnosis, challenge, repair, and verification. The original creative-system ambition makes this direction especially relevant: swarm coordination is valuable only if the underlying agents exchange typed evidence rather than mutually reinforcing plausible errors.

The historical project therefore loops back to its origin. The creative-system problem produced the debugger; the debugger produced an evidence and governance program; that program can eventually support more ambitious autonomous or multi-agent systems without discarding human authority and auditability.

15. Conclusion

AI Debugger Pro began because a different AI project exposed a deeper infrastructure problem. The attempt to assemble an autonomous creative production system revealed that generation alone was not the limiting factor: a larger system also needed dependable mechanisms for execution, diagnosis, repair, state tracking, verification, and orchestration.

The earliest debugger was intentionally simple: Python syntax checking, temporary-file execution, and terminal feedback. Subsequent versions added LLM interpretation, multi-language execution, a GUI, temporal history, structured repair proposals, and finally approval-gated mutation and verification. The research series then widened the question from 'can the debugger suggest a fix?' to 'what evidence justifies trusting any part of the repair process?'

Paper 0 is the bridge between those two eras. It preserves the origin story and the real V0-V4 development history, but it no longer presents early qualitative observations as if they were the current empirical record. The current record is narrower and stronger: a frozen execution benchmark, recorded diagnostic-evidence coverage, CI evidence, repository-grounded architecture, explicit security and verification models, typed context and oracle designs, and a reproducibility framework that distinguishes measurement from implementation and hypothesis.

The result is not a claim that AI Debugger Pro has solved autonomous software repair. It is a more defensible claim: the project has evolved from a lightweight debugging prototype into a structured research platform for studying human-governed, evidence-aware AI-assisted repair across multiple languages.

Recommended next reading: Paper 1 for the detailed V0-V6 evolution, Paper 2 for the authoritative current architecture, Paper 8 for the measured execution baseline, and Paper 12 for the reproducibility and release-evidence contract.

Appendix A. Evidence sources used by Paper 0

This overview synthesizes the original 22-page AI Studio Debugger document and the current repository research series. The original document supplies the project-origin anecdote, early V0-V4 architecture, and historical evaluation language. The current repository supplies the twelve-paper reading order and the modern claim boundaries.

Source	Use in Paper 0
Original 'AI Studio Debugger' PDF	Creative-studio origin; V0-V4 evolution; early architecture; historical qualitative evaluation and limitations
Paper-series catalog (docs/papers/README.md)	Unified twelve-paper titles, order, scope and historical-document status
Paper 8 measured-results.csv	24-case benchmark, 24 classification matches, latency, 14/16 evidence coverage, 27-test CI baselines
Paper 10 context data	Context-source priorities, privacy/injection risks, selection scenarios; treated as design specification
Paper 11 oracle data	Oracle independence ladder and regression-test admission scenarios; treated as design specification
Paper 12 claim/evidence data	Measured vs implemented vs hypothesis/planned status; reproducibility and release framing

Appendix B. Publication positioning

Suggested repository position: 00-foundation-overview/. The historical 'AI Studio Debugger.pdf' can be replaced in the public reading path by this paper while the original may be retained in an archive or version history if historical preservation is desired.

Suggested catalog title: 0. AI Debugger Pro: From Creative-System Failure to Evidence-Governed AI-Assisted Repair.

Paper 0 should precede Paper 1 because it explains why the project exists and how the twelve-paper program fits together; Paper 1 can then remain the detailed system-evolution case study rather than carrying the entire project narrative itself.